Creación y selección de personajes

Una guía visual para entender qué cambia al pulsar cada botón y cómo llega la elección hasta la granja.

Unity 6 · C# · UI Toolkit · Sin dependencias nuevas

El recorrido del jugador

Menú JUGAR › **Personalizar** nombre + tono › **Confirmar** validar nombre › **Guardar** PlayerPrefs › **Granja** aplicar tono

QUÉ PUEDES HACER

Escribir un nombre, elegir Natural, Bosque o Luna, ver una vista previa, escoger una de cinco plantillas o volver sin empezar.

QUÉ QUEDA GUARDADO

Nombre, índice del aspecto y número de plantilla. El tono se aplica al sprite del jugador al entrar en la granja.

QUÉ ES PROVISIONAL

Las cinco tarjetas usan el mismo sprite actual y nombres de ejemplo. Aún no son personajes finales.

ALCANCE HONESTO

El nombre y la plantilla se almacenan, pero todavía no se muestran ni se usan en la escena de juego.

Importante: la vista previa usa tintes de color; no se han creado nuevos sprites ni un sistema de personalización por partes.

La pantalla por dentro

El formulario se incorpora al UXML del menú. No se carga una escena nueva para mostrarlo: se alterna la visibilidad de dos paneles.



Distribución en el UXML

ZONA SUPERIOR

`creation-title` y `creation-subtitle` explican la pantalla.

ZONA DE EDICIÓN

`creation-layout` coloca `preview-panel` junto a `creation-form`.

ZONA DE PLANTILLAS

`presets-panel` contiene cinco botones y un elemento `preset-portrait` para el sprite.

ZONA DE ACCIONES

`creation-actions` contiene volver y confirmar.

Presentación y comportamiento

UXML define la jerarquía y los nombres. USS define tamaños, colores, imagen de fondo y clases de selección. C# consulta elementos mediante `Q<T>("nombre")` y registra los eventos. Las clases `appearance-button--selected` y `preset-card--selected` permiten que el estilo indique qué se eligió.

El boceto es una representación del layout implementado, no una captura del juego en ejecución.

Del clic al resultado

La pantalla y los scripts se conectan por identificadores UXML y callbacks de UI Toolkit.

1
PANELRENDERER
En `Awake` se registra `cargaArchivo`. El callback corre cuando la vista está lista.

2
BUSCAR CONTROLES
`root.Q<Button>("btn-play")` localiza el botón por su nombre UXML.

3
ENLAZAR EVENTOS
`clicked` y `RegisterValueChangedCallback` llaman la lógica C#.

Mapa de responsabilidades

MainMenu.uxml controles y nombres › **FuncionalidadBotonesInterface.cs** eventos y estado › **PlayerPrefs** elección persistida › **PlayerMovement.cs** tinte del sprite

Dos eventos importantes

AL EDITAR EL NOMBRE
El callback actualiza la vista previa y elimina la selección de tarjeta: el nombre ya no representa necesariamente esa plantilla.

AL ELEGIR UNA PLANTILLA
El código asigna `Aldeano N`. Ese cambio dispara el callback del campo; por eso el índice de plantilla se asigna después de cambiar el nombre.

Confirmación

```
nombre = campoNombrePersonaje.value.Trim()
si nombre está vacío → mostrar aviso y permanecer
si hay nombre → guardar 3 valores
PlayerPrefs.Save()
SceneManager.LoadScene(SceneVideojuego)
```

Al volver solo se alternan `DisplayStyle.Flex` y `DisplayStyle.None`; no se cambia de escena ni se elimina el progreso previamente guardado.

Qué se guarda y cómo se ve

El controlador guarda valores sencillos. La escena de granja consume directamente solo el índice de apariencia.

Clave PlayerPrefs	Tipo	Valor	Uso actual
`moonveil_personaje_nombre`	String	Nombre tras `Trim()`	Guardado; aún no se muestra en granja.
`moonveil_personaje_aspecto`	Int	0 Natural · 1 Bosque · 2 Luna	Leído por `PlayerMovement`.
`moonveil_personaje_predeterminado`	Int	-1 libre · 0 a 4 tarjeta	Guardado; no cambia el modelo en la granja.

Correspondencia del tono

Índice	Nombre	Color del SpriteRenderer	Resultado
0	Natural	Color.white	Color original.
1	Bosque	RGB 0.75, 0.92, 0.75	Tinte verde claro.
2	Luna	RGB 0.76, 0.80, 1.00	Tinte azul claro.

EN LA VISTA PREVIA

USS intercambia la clase `character-preview--N`; las tres reglas cambian el tinte del sprite de muestra.

EN LA GRANJA

C# lee el índice con valor por defecto 0, usa `Mathf.Clamp` y asigna el color al `SpriteRenderer`.

Límite técnico: la clave del aspecto es una constante en el controlador y un literal en `PlayerMovement`. Si cambia, hay que actualizar ambos lugares. Podría centralizarse en una clase compartida como mejora futura.

`PlayerPrefs` es almacenamiento sencillo de preferencias, no una base de datos ni un sistema de guardado de partidas. `PlayerPrefs.Save()` se llama antes de cargar la escena.

Cinco espacios preparados

Las tarjetas ya tienen botón, nombre e imagen provisional. Añadir personajes definitivos requiere conectar sus datos, no rediseñar desde cero la pantalla.

1

CREAR RETRATO

Reemplazar el sprite provisional del contenedor `preset-portrait`.

2

DEFINIR PLANTILLA

Asignar nombre y aspecto inicial propios en vez de generar `Aldeano N`.

3

SINCRONIZAR ÍNDICES

Conservar el índice guardado y actualizar los bucles si cambia el número de tarjetas.

Archivos y responsabilidad

`MainMenu.uxml`	Jerarquía, nombres de campos y cinco botones de plantilla.
`MainMenu.uss`	Distribución, fondo, colores y estilos de selección.
`FuncionalidadBotonesInterface.cs`	Eventos, estado, validación, guardado y carga de escena.
`PlayerMovement.cs`	Lectura del tono y cambio del color del sprite.
`EditorBuildSettings.asset`	Orden de inicio: menú antes que granja.
`README.md + PDF`	Guía de consulta: explicación textual y visual de la implementación.

Comprobaciones y límites

COMPROBADO

Compilación C# sin errores, UXML bien formado, controles y cinco tarjetas presentes. El PDF se genera desde HTML con impresión de Edge.

PENDIENTE EN UNITY

La interacción visual en Play Mode no se verificó manualmente. El layout debe probarse en distintas resoluciones.

Próximos pasos razonables: probar clics en Play Mode, incorporar sprites finales, mostrar el nombre en la granja y centralizar las claves de preferencias.

Comentarios añadidos en C# explican el callback de PanelRenderer, consultas UXML, captura de índices, selección de plantilla, validación/persistencia y aplicación del tinte.